

VTA-Simulator — Build, Run & Test Guide

A step-by-step manual for building the SystemC VTA simulator, running it on different inputs, and — most importantly — testing your own changes as you port `vta.h` / `vta.cc` (Vivado HLS) into SystemC.

Project: `E:\Uni Siegen\PEP\VTA-Simulator-main` • Platform: Windows 11 + VS Code + vcpkg

Read this first. There are only three actions you ever repeat: **Build** → **Run** → **Check the output**. Everything in this guide is a variation of those three. The hard one-time setup (compiler, libraries) is already done.

0. What you are actually doing

The files `vta.h` and `vta.cc` in the `PEP` folder are the original **VTA hardware design written in Vivado HLS C++** (they use `ap_int`, `hls::stream`, and `#pragma HLS` — Xilinx-specific). Your job is to re-express that same hardware behaviour as **SystemC** inside *this* simulator project, which already provides the matching module skeletons.

HLS function in <code>vta.cc</code>	becomes this SystemC module in the project	files
<code>fetch()</code>	<code>Fetcher</code>	<code>include/simulation/fetcher.h</code> , <code>src/simulation/fetcher.cpp</code>
<code>load()</code>	<code>LoadModule</code>	<code>include/simulation/load_module.h</code> , <code>src/simulation/load_module.cpp</code>
<code>compute()</code>	<code>ComputeModule</code>	<code>include/simulation/compute_module.h</code> , <code>src/simulation/compute_module.cpp</code>
<code>store()</code>	<code>StoreModule</code>	<code>include/simulation/store_module.h</code> , <code>src/simulation/store_module.cpp</code>
<code>vta()</code> (wrapper)	<code>VTA</code> (wires everything together)	<code>include/simulation/vta.h</code> , <code>src/simulation/vta.cpp</code>

So "transforming and testing" means: **edit the SystemC module files, rebuild, run the simulator, and confirm the output still makes sense** (e.g. all layers still finish, timings are reasonable).

1. Project layout (where everything lives)

Path	What it is
<code>main.cpp</code>	Entry point. Reads config, parses the model, starts the simulation.
<code>config/config.json</code>	The input control file — which model to run and all hardware settings.
<code>data/models/*.csv</code>	Input models (resnet_18 / 34 / 50 / 101). Each row = one VTA instruction.
<code>include/simulation/</code>	Module headers (the <code>.h</code> files you will edit).
<code>src/simulation/</code>	Module implementations (the <code>.cpp</code> files you will edit).
<code>CMakeLists.txt</code>	The build recipe. Lists every <code>.cpp</code> that gets compiled. You edit this when you add a new file.
<code>CMakePresets.json</code>	Tells CMake where the libraries are. Do not normally touch.
<code>build/</code>	Compiler output. Contains <code>my_systemc.exe</code> . Safe to delete; regenerated on build.
<code>build.cmd</code>	One-command build helper (terminal).

2. The core cycle: Build → Run → Check

A Build (compile the code)

In VS Code: press `Ctrl + Shift + P` → type `CMake: Build` → Enter. (If asked for a preset, choose `vcpkg`.) A clean build of everything is `CMake: Clean Rebuild`.

In a terminal (from the project folder):

```
.\build.cmd
```

Success looks like `==== BUILD OK =====` or, if nothing changed, `ninja: no work to do`. Either means you are good to run.

If the build fails, the terminal shows the file and line number of the first error (e.g. `compute_module.cpp(42): error C2065: ...`). Fix that line, build again. Always fix the *first* error first — later ones are often caused by it.

B Run (execute the simulation)

In VS Code: press `F5` and choose “Run VTA Simulator (Ninja build)”. Output appears in the terminal panel.

In a terminal:

```
.\build\my_systemc.exe
```

Why it just works: the two required DLLs (`libzmq-mt-4_3_5.dll` and `nlohmann_json_schema_validator.dll`) are copied next to the `.exe` in `build\` . If you ever delete `build\` and rebuild, re-copy them from `vcpkg\installed\x64-windows\bin\` (or just run from a terminal that has that folder on its `PATH`).

C Check the output

The simulator prints one line per network layer:

```
211040 ns      _____ FINISH LAYER 0 _____
--
758676 ns      _____ FINISH LAYER 1 _____
--
...
58021575 ns    _____ FINISH LAYER 102 _____
---
```

The number is the **simulated hardware time in nanoseconds** at which that layer finished. The difference between two consecutive lines is that layer's latency. This is the result you compare before and after a change.

3. Testing with different inputs

The "input" is chosen in `config/config.json`. The most important field is `model_name`.

Switch the model

1. Open `config/config.json`.
2. Change `"model_name"` to one of: `resnet_18`, `resnet_34`, `resnet_50`, `resnet_101`.
3. Save, then **Run** again (no rebuild needed — config is read at runtime).

Smaller models (resnet_18) have fewer layers and finish faster — handy for quick tests.

What every config field means

Field	Meaning / effect on the simulation
<code>model_name</code>	Which <code>data/models/<name>.csv</code> to simulate.
<code>granularity</code>	<code>"layer"</code> or <code>"block"</code> — how instructions are grouped/reported.
<code>en_cache</code>	Enable the on-chip cache model (changes memory timing).
<code>en_prefetch</code>	Enable prefetching.
<code>en_ddr_lock</code>	Model DRAM access locking/contention.
<code>max_cache_mem_size</code>	Cache size budget used by the cache model.
<code>nr_load_pipeline</code> / <code>nr_compute_pipeline</code> / <code>nr_store_pipeline</code>	How many parallel pipelines each stage has (more = more parallelism).
<code>load_cycles</code> / <code>load_weights_cycles</code> / <code>store_cycles</code>	Latency (in clock cycles) charged per load / weight-load / store operation. These directly drive the <code>ns</code> numbers.

Good test idea: run the same model twice — once with `en_cache=false` and once with `en_cache=true` — and compare the layer timings. This shows the cache model is having an effect, and is a clean way to demonstrate the simulator to a supervisor.

Make a tiny test model (fast iteration)

The full ResNet CSVs are large. To test a change quickly, copy a model and keep only the first few rows:

```
# PowerShell - create a 1-layer mini model from resnet_18
$src = "data\models\resnet_18.csv"
$dst = "data\models\mini.csv"
Get-Content $src -TotalCount 60 | Set-Content $dst # header + first ~59 instruction
rows
```

Then set `"model_name": "mini"` in the config. It runs in a fraction of a second, so you can build-run-check many times while debugging a module.

4. The transformation workflow (porting HLS → SystemC)

This is the loop you will repeat while moving logic from `vta.cc` into the SystemC modules.

1 Decide which module you are changing

Use the mapping table in section 0. E.g. porting the HLS `compute()` body → edit `include/simulation/compute_module.h` and `src/simulation/compute_module.cpp`.

2 Edit the SystemC code

Translate HLS idioms to SystemC equivalents:

HLS (vta.cc)	SystemC equivalent in this project
<code>hls::stream<T></code> (FIFO)	a <code>Queue</code> module connected by <code>vld/rdy/data/end</code> signals
<code>ap_int<N></code> / <code>ap_uint<N></code>	<code>sc_int<N></code> / <code>sc_uint<N></code>
<code>#pragma HLS PIPELINE</code> etc.	(ignore — pragmas are hardware-synthesis hints, not needed in simulation)
function called once, returns	module process driven by <code>sc_event</code> s, reacting over simulated time
dependence queues (<code>l2g</code> , <code>g2l</code> , <code>s2g</code> , <code>g2s</code>)	the <code>l2c</code> / <code>c2l</code> / <code>s2c</code> / <code>c2s</code> <code>Queue</code> s wired in <code>vta.cpp</code>

3 If you ADD a new `.cpp` file, register it in CMakeLists

The build only compiles files listed in `CMakeLists.txt`. Open it and add your new file inside the `add_executable(my_systemc ...)` list:

```
add_executable(my_systemc
    main.cpp
    src/utilities/utilities.cpp
    ...
    src/simulation/vta.cpp
    src/simulation/arm.cpp
    src/simulation/my_new_file.cpp    ← add your new .cpp here
)
```

Common mistake: creating a new `.cpp` but forgetting to add it here. Symptom: `unresolved external symbol` (linker error) at the end of the build. Fix = add the file to the list above. (Header-only changes to existing `.h` files do **not** need a CMakeLists edit.)

After editing `CMakeLists.txt`, run **CMake: Configure** once (VS Code does this automatically on save), then build.

4 Rebuild and run

Build (section 2A) → Run (section 2B). Watch for compile errors first, then runtime behaviour.

5 Verify the result (regression test)

Since the project has **no automated test suite**, you test by comparing output against a known-good baseline:

```
# 1) BEFORE your change - capture a baseline (do this once, on the working version):
.\build\my_systemc.exe > baseline.txt

# 2) AFTER your change - capture the new output:
.\build\my_systemc.exe > current.txt

# 3) Compare them:
Compare-Object (Get-Content baseline.txt) (Get-Content current.txt)
```

Interpretation:

- **No differences** → your change preserved behaviour (good for a refactor that shouldn't change results).
- **Timing numbers changed but all 0...N layers still finish** → expected if your change affects latency; check the new numbers are sensible.
- **Fewer layers, a crash, or it hangs** → a regression; your change broke something. Re-check the edited module.

5. A repeatable test checklist

Run through this every time you finish a transformation step:

#	Check	Pass criteria
1	It builds	<code>BUILD OK</code> / exit code 0, no errors
2	It runs without crashing	Process exits with code 0
3	All layers finish	You see <code>FINISH LAYER 0</code> ... up to the last layer of the model
4	Timings are monotonic & sensible	The <code>ns</code> values increase; no zero/negative/garbage values
5	Works on more than one model	resnet_18 and resnet_50 both complete
6	Matches baseline (if a pure refactor)	<code>Compare-Object</code> shows no unexpected differences

6. Troubleshooting

Symptom	Cause & fix
unresolved external symbol at link time	A <code>.cpp</code> isn't in <code>CMakeLists.txt</code> (section 4, step 3), or a function is declared but not defined.
App can't start: <code>...dll was not found</code>	Run from <code>build\</code> (DLLs are there), or add <code>vcpkg\installed\x64-windows\bin</code> to <code>PATH</code> , or re-copy the two DLLs into <code>build\</code> .
Build error <code>iostream: no include path set</code>	The compiler environment isn't loaded. Use <code>build.cmd</code> (it calls <code>vcvars64</code>) or build via VS Code's CMake Tools.
Error opening file: <code>...csv</code> / model not found	The CSV must be at <code>data/models/<model_name>.csv</code> . Check the name in <code>config.json</code> matches a file there.
CMake can't find SystemC / a library	The vcpkg preset isn't selected. In VS Code choose the vcpkg configure preset; or re-run <code>build.cmd</code> .
Changed config but output is identical	You may have edited a copy. Confirm you saved <code>config/config.json</code> in the project root.

7. Quick reference (cheat sheet)

```
# Build (terminal)
.\build.cmd

# Run
.\build\my_systemc.exe

# Run and save output to a file
.\build\my_systemc.exe > run.txt

# Switch model: edit config/config.json → "model_name": "resnet_18"

# Add a new source file: add it to add_executable(...) in CMakeLists.txt, then
rebuild

# Compare two runs (regression test)
Compare-Object (Get-Content baseline.txt) (Get-Content current.txt)
```

The one-sentence summary: edit the SystemC module files → `build.cmd` (or VS Code Build) → `my_systemc.exe` → confirm all layers finish with sensible `ns` timings, comparing against a saved baseline.